API Integration Differences with PWA Kit

While both PWA Kit and Storefront Next use SCAPI for commerce functionality, they take fundamentally different approaches to API client architecture.

Architecture

Aspect	PWA Kit	Storefront Next
Package	`@salesforce/commerce-sdk-react`	`@salesforce/s next-runtime/`
Underlying Client	`commerce-sdk-isomorphic`	`openapi-fetch`
Pattern	React hooks	Direct API calls
Type Generation	Manual TypeScript	Generated from specs
Bundle Size	Larger (includes React Query + SDK)	Smaller (lightwe

API Call Location

Aspect	PWA Kit	Storefront Next
Reads (GET)	Inside components via hooks	In route loaders
Writes (POST/PUT)	Inside components via mutation hooks	In action routes
Authentication	Automatic via provider	Manual via middleware

Client Initialization

Where	`CommerceApiProvider`	`createApiClien` helper
Auth	Automatic token management	Middleware injects tokens

Caching

Aspect	PWA Kit	Storefront Next
Strategy	React Query cache	React Router revalidation
Invalidation	Automatic after mutations	Manual or via `shouldRevalidate`
Stale Time	Configurable (default 10s)	No built-in stale time

Code Examples Comparison

Fetching a product:

```
// PWA Kit - in component
const { data: product, isLoading } = useProduct(
  parameters: { id: productId },
});
```

```
// Storefront Next - in loader
const { data } = await clients.shopperProducts.g
  params: {
    path: { organizationId, id: productId },
    query: { siteId },
  },
});
```

Adding to Cart:

```
// PWA Kit - in component
const addItem = useShopperBasketsMutation("addIt
addItem.mutate({
  parameters: { basketId },
  body: { productId, quantity },
});
```

```
    path: { organizationId, basketId },
    query: { siteId },
  },
  body: [{ productId, quantity }],
});
```

Key Similarities

- Both use SCAPI for commerce operations.
- Both support SLAS authentication.
- Both proxy API requests through the server.
- Both provide TypeScript support.

Storefront Conversion Considerations

When migrating from PWA Kit to Storefront Next:

1. Move API calls to loaders: Replace `useProduct`, `useCategory`, and so on with loader fetches.
2. Move mutations to actions: Replace `useShopperBasketsMutation` with action routes.
3. Update parameter format: Change from `{ parameters: {} }` to `{ params: { path: {}, query: {} } }`.
4. Handle auth manually: Create auth middleware instead of relying on provider.
5. Remove React Query: No longer needed; caching handled by React Router.
6. Update error handling: Use `ApiError` class instead of hook error states.

Try for free

Quip